



“ARTICOLE SPORTIVE DE VARF”



Autor: IonutD

CUPRINS

Capitolul 1	Introducere
Capitolul 2	Tema de proiect
Capitolul 3	Schema conceptuala
3.1.	Notiuni teoretice
3.2.	Schema conceptuala
Capitolul 4	Schema logica
4.1.	Notiuni teoretice
4.2.	Schema logica
Capitolul 5	Normalizarea bazei de date
5.1.	Notiuni teoretice
5.2.	Normalizarea bazei de date
Capitolul 6	Denormalizarea bazei de date
6.1.	Notiuni teoretice
6.2.	Denormalizarea bazei de date
Capitolul 7	SGBD-uri
7.1.	Notiuni teoretice
7.2.	Aplicatii
Capitolul 8	Concluzii

Capitolul 1

Introducere

SQL a fost conceput ca un limbaj standard de descriere a datelor și acces la informațiile din baze de date, dezvoltându-se ca o adevărată tehnologie dedicată arhitecturilor client/server.

Utilizat inițial de către firma IBM pentru produsul DB2, limbajul de interogare a bazelor de date relaționale SQL (Structured Query Language) a devenit la mijlocul deceniului

trecut un standard in domeniu. De atunci și pînă în prezent au fost dezvoltate mai multe versiuni ale standardului SQL, trei dintre ele aparținînd Institutului National American de Standarde (ANSI), celelalte fiind concepute de firme de prestigiu ca IBM, Microsoft, Borland. Din păcate, lipsa unui standard unic SQL are drept consecințe creșterea costurilor programelor de gestiune a bazelor de date și îngreunează întreținerea arhitecturilor client/server. Primul standard SQL a fost creat în anul 1989 de către ANSI, fiind cunoscut sub numele de ANSI-SQL '89 și a fost revizuit în 1992 sub noua denumire ANSI-SQL '92. Proliferarea între timp a diferitelor implementări ale limbajului SQL, a determinat formarea consorțiului SAG, cu scopul de a concepe un set de standarde SQL, care să aibă la baza un subset al standardului ANSI-SQL '89 și în plus să rezolve problemele legate de implementarea acestora în arhitecturi client/server cum ar fi conectivitatea și interfețele cu utilizatorul ale programelor.

În anul 1992, firma Microsoft, în calitate de membru SAG a lansat pe piața produsul ODBC (Open Database Connectivity), un standard API-SQL ce are la baza un proiect de standard și definește o interfață de programare a aplicațiilor (API) pentru accesul la baze de date.

Interfața de programare cunoscută în domeniu sub inițialele API, oferă o cale de comunicare între programe și bazele de date prin apeluri de funcții care substituie instrucțiunile SQL. Astfel, standardele API se bazează pe conectivitate, precum și schimbul de date și mesaje SQL.

SQL este un limbaj complet pentru baze de date, cuprinzînd comenzi pentru definirea datelor, actualizare și interogare. Deci, este atât un limbaj pentru definirea datelor (DDL), cit și pentru manipularea datelor (DML). În plus, are facilități pentru definirea viziunilor, pentru crearea și ștergerea indecșilor și pentru încapsularea comenzilor SQL în limbaje de programare ca C sau Pascal.

Comenzile SQL pentru definirea datelor sunt CREATE, ALTER și DROP.

Comenzile SQL pentru modificarea datelor sunt INSERT, DELETE și UPDATE.

SQL are o singură comandă pentru regăsirea datelor din bazele de date, SELECT .

Avînd în vedere că în ASP, accesul la bazele de date de tip Access se realizează prin intermediul limbajului SQL, voi prezenta în continuare, sintaxa comenzilor SQL în Microsoft Access.

1.1 Comanda SELECT

Permite returnarea unui set de înregistrări din baza de date. Are următoarea sintaxă:

```
SELECT [predicate] { * | table.* | [table.]field1 [AS alias1]
```

```
[,table.]field2 [AS alias2] [, ...] }
```

```
FROM tableexpression [, ...] [IN externaldatabase]
```

```
[WHERE... ]
```

```
[GROUP BY... ]
```

```
[ORDER BY... ]
```

Comanda SELECT are următoarele părți:

- Predicate – poate fi unul dintre următoarele predicate: ALL, DISTINCT, DISTINCTROW, or TOP, care se utilizează pentru a restricționa numărul de înregistrări returnate. (implicit este ALL).

- * - specifica selectarea tuturor câmpurilor din tabela sau tabelele selectate.
- table – specifica numele tablei continind câmpurile din care se selectează datele
- field1, field2 – reprezintă numele câmpurilor din care dorim regăsirea datelor.
- alias1, alias2 - reprezintă numele utilizate ca și titluri de coloane in locul numelor coloanelor din tabela
- tableexpression – reprezintă numele tablei sau tabelelor din care se regasesc datele
- externaldatabase – reprezinta numele bazei de date externe, daca tabelele nu sint in cea curenta

Observatii: Pentru a executa aceasta operatie, motorul de baze de date Microsoft Jet cauta tabela sau tabelele specificate, extrage coloanele alese, selecteaza rindurile care satisfac criteriul și sorteaza sau grupeaza rindurile rezultate in ordinea specificata.

Comanda SELECT nu modifica datele din baza de date.

Daca numele unui cimp apare in mai multe tabele din clauza FROM, el trebuie precedat de numele tablei și operatorul punct (.).

Clauza WHERE specifica inregistrările din tabelele ce apar in clauza FROM care vor fi afectate de comanda SELECT. Numai acele inregistrari care satisfac criteriul din aceasta clauza vor fi incluse in interogarea rezultat.

Observatii: Daca nu se specifica clauza WHERE vor fi returnate toate rindurile din tabele. Daca in interogare se specifica mai mult de o tabela și nu specifica clauza WHERE sau JOIN, se efectueaza produsul cartezian al tabelelor, operatie consumatoare de timp.

Clauza ORDER BY permite sortarea inregistrărilor dupa valorile din cimpurile specificate, ascendent sau descendent.

Sintaxa acestei clauze este: [ORDER BY field1 [ASC | DESC][, field2 [ASC | DESC]][, ...]]

Observatii: Clauza este optionala. Sortarea implicita este ascendenta, altfel trebuie specificat DESC. In aceasta clauza nu pot apare cimpuri de tip OLE sau Memo.

Clauza GROUP BY combina inregistrările cu valori identice din lista de cimpuri specificate, intr-o singura inregistrare. Daca in comanda SELECT se specifica una dintre functiile agregat (Sum, Count, Max), atunci pentru fiecare astfel de inregistrare se creaza apoi o valoare agregat.

Clauza are urmatoarea sintaxa:

[GROUP BY groupfieldlist]

- groupfieldlist – reprezinta numele a cel mult 10 cimpuri utilizate pentru gruparea inregistrărilor. Ordinea acestor cimpuri in lista determina nivelele de grupare de la cel mai inalt pina la cel mai de jos

Observatii:

Clauza este optionala. Valorile agregat sint omise daca in comanda SQL nu se specifica nici o functie agregat. Nu se poate face grupare dupa cimpuri de tip Ole sau Memo.

Jonctiunea interna (INNER JOIN)

Se specifica cu urmatoare sintaxa:

FROM table1 INNER JOIN table2 ON table1.field1
compopr table2.field2

Operatia INNER JOIN are urmatoarele parti:

- table1, table2 - reprezinta numele tabelelor din care se combina inregistrările
- field1, field2 – reprezinta numele cimpurilor dupa care se face jonctiunea. Daca ele nu sint numerice, trebuie să fie de același tip, aceeași dimensiune, dar nu trebuie să aiba același nume.

- compopr – reprezinta orice operator relational: "=", "<," ">," "<=," ">=," sau "<>."
Observatii:

Acest tip de jonctiune este cel mai utilizat și se poate folosi in orice clauza FROM. Combina inregistrările din doua tabele, ori de cite ori exista valori care își corespund in cimpul comun din cele doua tabele.

Nu se poate face jonctiune pe tipuri de date OLE sau Memo.

1.2. Comanda UPDATE

Permite modificarea valorilor cimpurilor intr-o tabela specificata, pe baza unui criteriu specificat. Are urmatoarea sintaxa:

```
UPDATE                                     table
SET                                       newvalue
WHERE criteria
```

Comanda UPDATE are urmatoarele parti:

- table – reprezinta numele tablei ce contine datele ce vor fi modificate.
- newvalue - este expresia care determina valoarea ce va fi memorata intr-un anume cimp din inregistrările care se actualizeaza.
- criteria – este expresia care determina inregistrările ce vor fi actualizate

1.3. Comanda INSERT

Permite adaugarea unei sau mai multor inregistrari intr-o tabela.

Pentru adaugarea unei singure inregistrari se foloseste urmatoarea sintaxa:

```
INSERT      INTO      target      [(field1[,      field2[,      ...]])]
VALUES (value1[, value2[, ...])]
```

Comanda are urmatoarele parti:

- target – reprezinta numele tablei sau interogarii in care se insereaza date
- field1, field2 – reprezinta numele cimpurilor pentru care se furnizeaza date
- value1, value2 – reprezinta valorile care se insereaza in noua inregistrare. Value1 se insereaza in field1, value2 in field2, s.a.m.d. Valorile se separa prin virgula, iar valorile de tip text se incadreaza intre apostrofuri.

1.4. Comanda DELETE

Permite stergerea inregistrărilor din una sau mai multe tabele precizate in clauza FROM, inregistrari care satisfac clauza WHERE. Are urmatoarea sintaxa:

```
DELETE [table.*] FROM table WHERE criteria
```

Comanda DELETE are urmatoarele parti:

- table – numele tablei din care se vor sterge inregistrările
- criteria – expresia care determina inregistrările de sters

Se numește *bază de date* o colecție de date conectate logic exhaustivă, neredundantă și care suportă independența aplicațiilor în raport cu structura datelor.

Prin *structura bazei de date* se materializează de fapt informația conținută de date. Structura datelor nu este unică. Prin duplicarea datelor înțelegem că aceleași date sunt reprezentate în baza de date în aceeași formă sau în forme apropiate.

Conexiunea logică a datelor presupune că în baza de date sunt introduse numai date specifice unui anumit domeniu iar între ele pot fi puse în evidență legături cu o semantică bine determinată.

Exhaustivitatea datelor se referă la cerința impusă de precizia informațiilor și anume că pentru a obține informații cât mai corecte într-un anumit domeniu respectiv.

Prin *cerere* vom înțelege o solicitare abstractă sistemului informatic sub forma unor comenzi într-un limbaj de programare în scopul obținerii unui anumit rezultat cu ajutorul unor prelucrări elementare asupra datelor păstrate în baza de date.

Prin *aplicație* se înțelege un grup de cereri concatenate semantic și care solicită aproximativ aceleași date pentru obținerea rezultatelor. În concluzie există un număr mare de elemente ale bazei de date a căror modificare nu trebuie să afecteze aplicațiile deja implementate. Printre acestea putem aminti:

- utilizarea unor tipuri de dată diferite pentru valorile aceluiași atribut
- formatul de reprezentare al datei calendaristice poate varia foarte puternic în funcție de aplicație, de preferințele beneficiarului de tranzacție.
- anumite date pot fi văzute distinct de către o aplicație și concatenate de către altă aplicație.
- dacă la un moment dat se hotărăște extinderea bazei de date prin adăugarea de noi categorii de informații, vechile aplicații care nu utilizează aceste date nu trebuie să fie afectate.
- având în vedere prețul de cost mare al implementării unei baze de date și a aplicațiilor peste aceasta se poate decide implementarea bazei de date în etape.
- modificările în structura sistemului informatic prin achiziția de echipamente de calcul noi prin modificarea tipului suportului extern de informație, prin redistribuirea fișierelor de date nu trebuie să afecteze programele aplicative.
- un program aplicativ poate fi utilizat în contexte diferite peste baze de date cu structuri de date identice dar implementate pe sisteme de calcul diferite.

Capitolul 2

Tema de proiect

Articole sportive de varf

Sunt manager al unei companii en-gros de articole sportive care operează la nivel mondial pentru a completa comenzile magazinelor de articole sportive de specialitate.

Magazinele sunt clienții noștri (unii dintre oamenii noștri preferă să le numească clienții noștri).

Acum avem cincisprezece clienți la nivel mondial, dar încercăm să ne extindem baza de clienți cu aproximativ 10% în fiecare an începând cu acest an.

Cei mai mari doi clienți ai noștri sunt Big John's Sports Emporium din San Francisco, CA, SUA și Womansports din Seattle, Washington, SUA. Pentru fiecare client trebuie să urmărim un ID și un nume.

Noi putem să urmărim o adresă (inclusiv orașul, statul, codul poștal și țara) și numărul de telefon.

Mentinem depozite în diferite regiuni pentru a completa cel mai bine comanda clientilor noștri.

Pentru fiecare comandă trebuie să urmărim un ID.

Putem urmări data comenzii, data expedierii și tipul de plată atunci când informațiile sunt disponibile.

Acum avem lumea împărțită în cinci regiuni: America de Nord, America de Sud, Africa / Orientul Mijlociu, Asia și Europa.

Urmărim doar ID-ul și numele.

Încercăm să alocăm fiecărui client unei regiuni, astfel încât vom cunoaște în general cea mai bună locație din care să umplem fiecare comandă.

Fiecare depozit trebuie să aibă un ID.

Este posibil să urmărim o adresă (inclusiv orașul, statul, codul poștal și țara) și numărul de telefon.

În prezent avem un singur depozit pe regiune, dar sperăm să avem mai multe curând. "

"Administrez funcțiile de înscriere a comenzii pentru afacerea noastră de articole sportive en-gros.

Departamentul meu este responsabil pentru plasarea și urmărirea comenzilor la apelurile clienților noștri.

Pentru fiecare departament trebuie să urmărim ID-ul și numele.

Uneori, clienții noștri ne trimit prin e-mail comenzile atunci când nu se grăbesc, dar cel mai adesea ne sună sau ne trimit prin fax o comandă.

Sperăm să ne extindem afacerile oferind clienților noștri o schimbare imediată a informațiilor despre comenzi.

Credeți că această aplicație poate fi pusă pe Web?

Putem promite să expediem până a doua zi, atât timp cât mărfurile sunt în stoc (sau inventar) la una dintre locațiile depozitului nostru.

Când informațiile sunt disponibile, urmărim cantitatea din stoc, punctul de reordonare, stocul maxim, un motiv pentru care ne rămânem din stoc și data la care am reîncărcat articolul.

Atunci când mărfurile sunt expediate, intenționăm să trimitem prin fax informațiile de expediere automat prin sistemul nostru de transport.

Nu, nu administrez acea zonă.

Departamentul meu doar se asigură că clienții noștri au informațiile de facturare corecte și verifică dacă contul lor este în stare bună de credit.

De asemenea, putem înregistra comentarii generale despre un client. "

Ne asigurăm că toate articolele solicitate sunt în stoc.

Pentru fiecare articol urmărim un ID.

De asemenea, este posibil să urmărim prețul, cantitatea și cantitatea expediata, dacă informațiile sunt disponibile.

Dacă sunt în stoc, dorim să prelucrăm comanda și să le spunem clienților noștri care este ID de comandă și cât este totalul comenzii lor.

Dacă mărfurile nu sunt în stoc, clientul ne spune dacă ar trebui să reținem comanda pentru o expediere completă sau să procesăm comanda parțială.

"

Departamentul de contabilitate este responsabil pentru păstrarea informațiilor despre clienți, în special pentru atribuirea de noi ID-uri pentru clienți.

Departamentul meu are voie să actualizeze informațiile despre clienți numai atunci când este plasată o comandă și se schimbă adresa de facturare sau de expediție.

Nu, nu suntem responsabili pentru colecții.

Toate sunt gestionate de conturile de primire, de asemenea, cred că reprezentanții de vânzări se implică, deoarece comisionul lor depinde de clienții care plătesc!

Pentru fiecare reprezentant de vânzări sau angajat, trebuie să cunoaștem ID-ul și prenumele.

Ocazional, trebuie să cunoaștem numele, ID-ul utilizatorului, data de începere, titlul și salariul.

De asemenea, putem urmări procentul de comision al angajaților și orice comentarii despre persoana respectivă. "

"Personalul nostru de la intrarea comenzilor este bine versat în linia noastră de produse.

Ținem întâlniri frecvente cu departamentul de marketing, astfel încât să ne poată informa despre noi produse.

Acest lucru duce la o satisfacție mai mare a clienților, deoarece operatorii noștri de intrare în comenzi pot răspunde la multe întrebări.

Acest lucru este posibil deoarece avem de-a face cu câțiva clienți selectați și menținem o linie de produse de specializare.

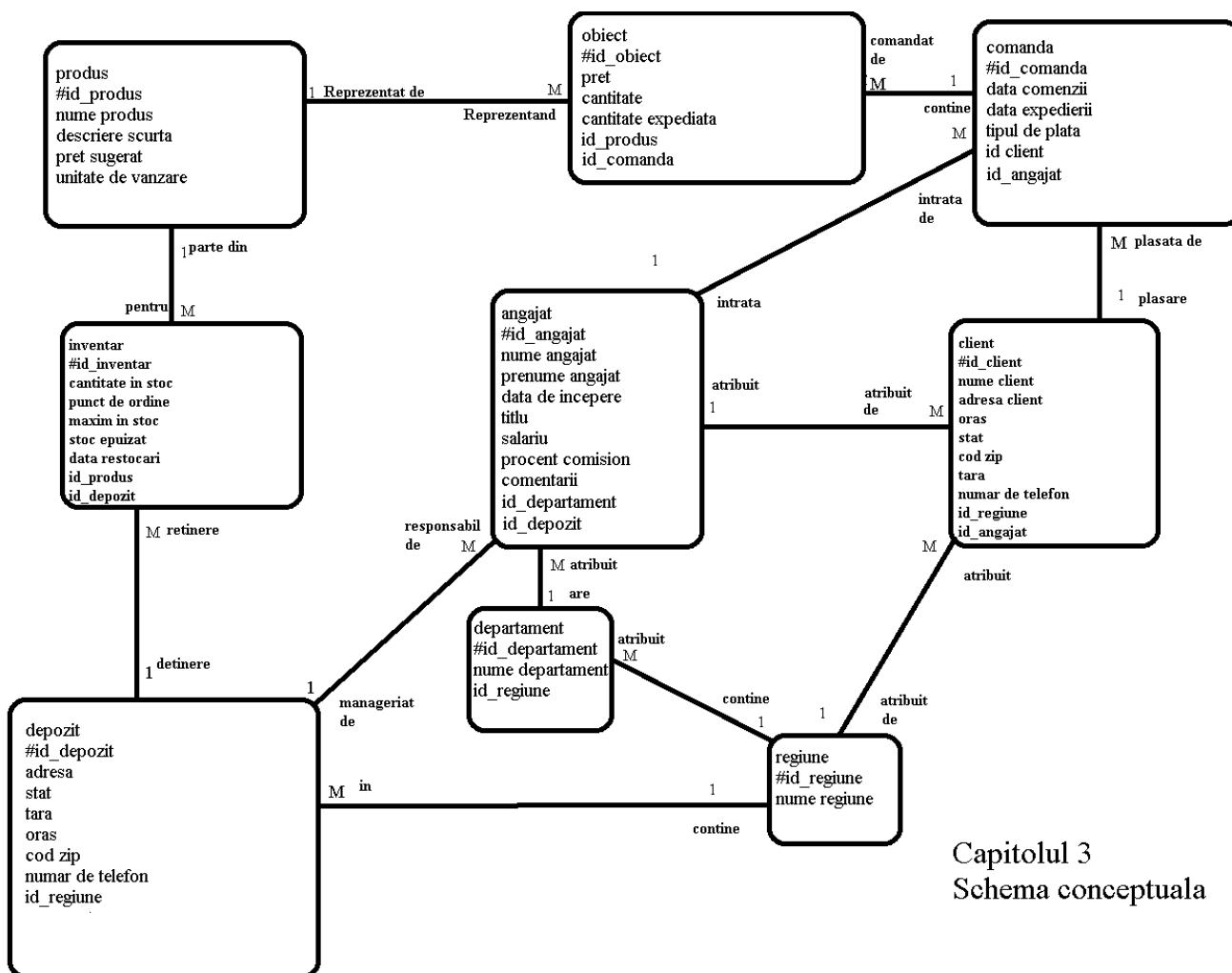
Pentru fiecare produs trebuie să cunoaștem ID-ul și numele.

Ocazional, trebuie să cunoaștem și descrierea, prețul sugerat și unitatea de vânzare.

De asemenea, am dori capacitatea de a urmări descrieri lungi ale produselor și imagini ale produselor noastre, atunci când este necesar. "

Capitolul 3

Schema conceptuala



Capitolul 3
Schema conceptuala

Capitolul 4 Schema logica

4.1. Notiuni teoretice

Se numește *entitate* orice obiect, fenomen sau concept care poate fi distins prin valorile caracteristicilor sale de alte obiecte, fenomene sau concepte asemănătoare.

Se numește *atribut* o caracteristică specifică unei entități.

Un atribut se caracterizează prin :nume,semantică.structură,tipul datelor și domeniul de definiție.

Prin *tipul datelor* înțelegem structura datei,mulțimea valorilor,operațiile admise și modul de tratare a erorilor specifice.

Semantica înseamnă semnificația atributului în cadrul entității.Într-o colecție pot să apară diverse atribute cu același nume dar plasate în entități diferite.Chiar dacă au același nume atributele pot să aibă semnificații diferite.

Domeniul este o noțiune specifică bazelor de date și puternic legată de semantica acestora.Cunoașterea semanticii atributelor și a domeniilor specifice este una din cerințele de bază ale proiectării unei baze de date.

Din punct de vedere al semanticii distingem :

- *Attribute identifier*-ale cărui valori pot identifica în mod unic o anumita entitate.
- *Attribute descriptor*-ale carui valori descriu o anumită proprietate a entității.
- În asemenea situații o entitate poate să nu aibă cheie primară naturală.În acest caz proiectantul bazei de date poate decide introducerea unei chei primare artificiale.
- Din punct de vedere al structurii putem distinge :
- *Attribute atomic* din a căror valoare nu poate fii eliminată nici o componentă fără a afecta semantica atributului.
- *Attribute compuse* a căror valoare este dată de reuniunea valorilor unor atribute atomice.
- Din punct de vedere al numărului de valori pe care-l poate lua la un moment dat un atribut pentru o anumită entitate distingem.
- *Attribute cu valoare unică* care acceptă la un moment dat o singură valoare pentru fiecare entitate.
- *Attribute multivaloare* care acceptă la un moment dat un set de valori pentru aceeași entitate.
- Din punct de vedere al nivelului de structurare al unui atribut distingem :
- *Attribute slab structurate* –sunt caracteristice aplicațiilor multimedia.
- *Attribute puternic structurate*-sunt atribute care au o structură riguros definită.

Între entități se stabilesc legături.Legătura între clase de entități pune în corespondență entitățile ce participă la realizarea legăturii a două sau mai multe clase de entități reprezintă o submulțime a produsului cartezian al entităților.

Caracteristici generale ale legăturilor:

- Orice legătură are un nume
- Orice legătură are o semantică riguros definită

- legătură este bidirecțională
- Pot fi definite legături între entitățile aceleiași clase de entități
- Între aceleași clase de entități pot fi definite mai multe legături cu semnificații diferite
- Pot fi definite legături peste mai mult de două clase de entități
- Legăturile pot avea atribute proprii ca și entitățile
- Indiferent de semantica lor legăturile prezintă câteva proprietăți deosebit de importante a căror cunoaștere este esențială pentru înțelegerea și implementarea colecțiilor de date.

Gradul legăturii-indică numărul de clase de entități care participă la realizarea legăturii tip .

Indicator de participare-este un indicator care specifică dacă pot exista entități ale unei clase de entități care să nu participe obligatoriu la realizarea legături tip analizate.

Indicator de conectivitate-indică numărul de entități ale unei clase care pot fi puse în corespondență cu entități ce participă la realizarea unei anumite legături.

CHEI

Noțiunea de cheie derivă din conceptul determinant .

Se numește *determinant* orice atribut simplu sau compus ale cărui valori identifică în mod unic valorile altor atribute.

Se numește *supercheie* orice atribut simplu sau compus care identifică în mod unic un tuplu.

Există o supercheie care apare în orice relație numită cheie trivială și anume o supercheie formată din toate atributele relației.

1. Cheie candidat :

Se numește cheie candidat o supercheie minimală adică o supercheie din care îndepărtând cel puțin un atribut se pierde caracterul de determinant. Supercheile și cheile candidat rezultă în mod natural din semantica datelor.

2. Cheie primară :

Cheia primară este o cheie candidat aleasă de către proiectantul bazei de date pentru identificarea tuplurilor într-o relație. Specificarea cheii primare este obligatorie în orice documentație privind baza de date. Alegerea cheii primare se poate face pe baza unei analize profunde a semnificației datelor și a modului în care acestea sunt utilizate în diverse aplicații sau pot fi fixate arbitrar dintre cheile candidat. În cazul în care nu există nici o cheie candidat sau cheile candidat naturale au un număr prea mare de atribute proiectantul poate decide introducerea unor chei primare artificiale generate de către SGBD.

3. *Cheie alternantă* : este o cheie candidat care nu a fost aleasă pentru postul de cheie primară.

4. Cheie secundară :

Orice atribut sau grup de atribute prin ale căror valori se identifică o mulțime de tupluri dintr-o relație.

Fie o relație având schema $r(a_1, a_2, \dots, a_n)$, unde a_1 este cheie primară a relației. Se numește domeniu primar al relației domeniul corespunzător cheii primare. Fie o altă relație $s(b_1, b_2, \dots, b_n)$, unde b_1 este definit peste același domeniu primar din relația r . În acest caz se spune că b_1 este cheie externă peste definită peste cheia primară din relația r .

5. *Cheia externă* reprezintă un mecanism fundamental pentru materializarea legături între relațiile r și s . Cheile externe pot fi definite în practică peste chei candidat în afara cheii primare.

Reguli de integritate

Regulile de integritate a datelor dintr-o bază de date relațională reprezintă restricții impuse datelor cu scopul de a elimina redundanța datelor, inconsistență a datelor și erori de actualizare a datelor.

Există multe tipuri de integrități și anume :

a. *Integritatea domeniului*-această restricție presupune definirea domeniului unui atribut astfel încât să reprezinte efectiv valorile posibile ale unui atribut.

b. *Integritatea cheilor*-prin cheie vom înțelege toate cheile candidat puse în evidență într-o relație.

c. *Integritatea entității*-prin definiție cheia primară trebuie să identifice în mod unic un tuplu al unei relații care în general este materializarea unei entități. Pentru ca să-și poată realiza rolul de identificator cheia primară nu poate avea valori nule.

d. *Integritatea referirii*-această restricție de integritate are ca scop să elimine referirile în gol, adică cheia externă nu are voie să se refere la un tuplu care nu există. Restricția integrității referirii impune ca la un moment dat valorile cheii externe fie să fie nule, fie să se afle printre valorile deja înregistrate pentru cheia primară peste care este definită cheia externă.

Restricții de utilizator-în unele situații utilizatorul poate impune și alte restricții decât cele impuse de regulile de integritate discutate anterior :

- unicitatea altor atribute decât cheia primară
- valori nule
- diverse validări

Operatori ai algebrei relaționale

Operatorii algebrei relaționale pot fi împărțiți în două grupe:

- Operatori clasici peste mulțimi
- Operatori relaționali specifici

Se spune că relațiile r_1 și r_2 sunt compatibile cu reuniunea dacă cele 2 relații au aceeași paritate și atributele care ocupă aceeași paritate sunt definite peste același domeniu chiar dacă au nume diferite.

Reuniunea : Fie r_1 și r_2 două relații compatibile cu reuniunea. Se numește reuniune relația r care conține toate tuplurile celor 2 relații inițiale fără duplicate.

Intersecția : Fie r_1 și r_2 două relații compatibile cu reuniunea. Se numește intersecție relația r care conține toate tuplurile comune lui r_1 și r_2 .

Diferența : Fie r_1 și r_2 două relații compatibile cu reuniunea. Se numește diferență relația r care conține numai tuplurile din r_1 care nu apar în r_2 .

Produsul cartezian : Fie r_1 și r_2 două relații oarecare. Se numește produs cartezian relația r obținută prin concatenarea fiecărui tuplu din r_1 cu fiecare tuplu din r_2 .

Operatori relaționali specifici :

Selecția-Fie o relație r și o expresie relațională aritmetică f numită filtru. Se numește selecție o relație formată din toate tuplurile relației r pentru care atributele iau valori care înlocuite în expresia f fac ca aceasta să aibă valoarea adevărată true.

Proiecția-Fie o relație r și $a_1, a_2, \dots, a_n$, un set de atribute ale acestei relații. Se numește proiecție o relație obținută din relația r care conține numai atributele specificate.

Tipuri de limbaje utilizate în domeniul bazelor de date

Limbaje de descriere a bazei de date (LDBD)-permit descrierea relațiilor atributelor și a unor restricții de integritate, a unor vederi asupra bazelor de date a unor utilizatori și a privilegiilor acordate acestora.

Limbaje de manipulare a bazelor de date (LMDB)-acestea cuprind comenzi care permit :

- introducerea de noi date în baza de date

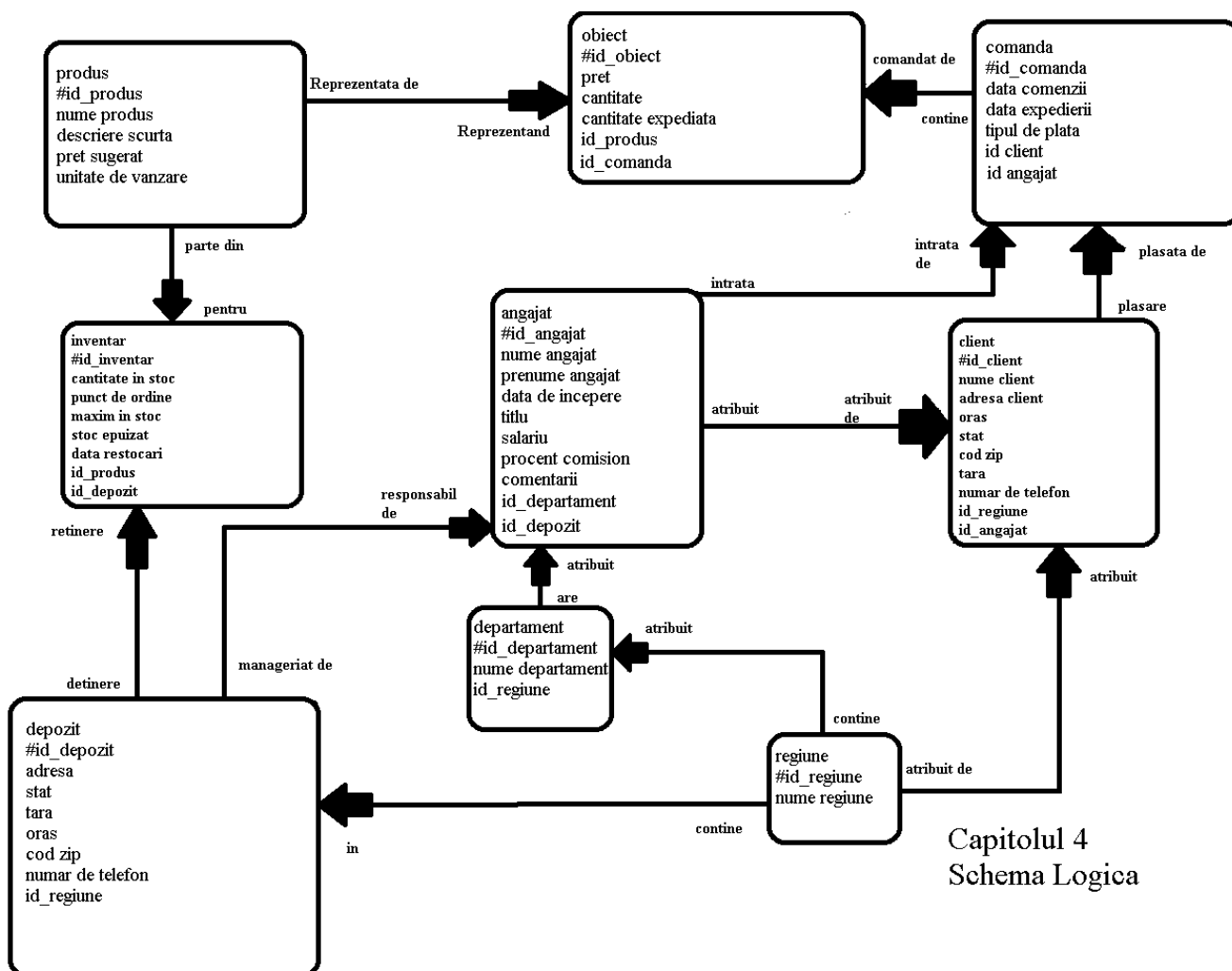
- modificare valorilor deja înscrise în baza de date

- eliminarea unor tupluri ale unei relații

Limbaje de interogare-au ca scop posibilitatea formulării unor cereri de căutare a informațiilor în baza de date.

Limbaje pentru generarea rapoartelor-sunt limbaje care permit o

formatare sofisticată a datelor care să prezinte informația într-o formă adecvată problemei rezolvate.



Capitolul 5

Normalizarea bazei de date

5.1. Notiuni teoretice

Este un proces interativ și reversibil prin care se înlocuiește o colecție dată de relații prin colecții succesive în care relațiile au o structură din ce în ce mai simplă și mai regulată.

Obiectivele normalizării sunt :

- 1.Să permită reprezentarea oricărei relații în baza de date .
- 2.Să obțină algoritmi de căutare puternici bazați pe o colecție de operatori relaționali .
- 3.Să elimine din relații anomaliile de inserție ,modificare și ștergere.
- 4.Să reducă necesitatea restructurării relațiilor atunci când se introduce un nou tip de date sau se modifică ipotezele de lucru.
- 5.Să facă colecția de relații imună în raport cu cererile aleatoare a căror semantică se modifică în timp.

Pot fi puse în evidență cinci forme de normalizare ce formează o structură riguros ierarhizată care vor fi notate cu FN1-FN5.

Forma normală 1(FN1)

O relație este în FN1 dacă fiecare atribut al relației are un domeniu de valori atomice.FN1 este cea mai slabă formă normală.În general relațiile în această formă prezintă numeroase anomalii la inserție,modificare și eliminare, acestea fiind eliminate de celelalte forme normale.

Forma normală 2(FN2)

Fie X mulțimea tuturor atributelor relației $r(a_1, a_2, \dots, a_n)$, care nu participă la formarea vreunei chei în r . Se spune că relația r este în FN2 dacă și numai dacă fiecare atribut din X este complet dependent funcțional de oricare din cheile lui r . Un atribut este *atribut prim* dacă face parte din cel puțin o cheie a relației r . Un atribut este *atribut nonprim* dacă nu face parte din cel puțin o cheie a relației r . O relație este în FN2 dacă și numai dacă orice atribut nonprim din r este complet dependent funcțional de toate cheile din r .

Proprietate :

Dacă o relație nu este în FN2, există o descompunere a lui r într-un set de proiecții, fiecare în FN2. Descompunerea se poate realiza în așa fel încât aplicând joncea naturală între proiecții se obține conținutul informațional al lui r fără pierdere de informație.

Forma normală 3(FN3)

O relație este în FN3 dacă și numai dacă nici unul din attributele sale nonprime nu este dependent tranzitiv de o cheie a relației r .

Proprietăți :

1. Dacă o relație se află în FN3 ,ea este obligatoriu în FN2.
2. Există relații în FN2 care nu se află în FN3.
3. Dacă o relație r nu este în FN3, există o decompoziție a lui r într-un set de proiecții, toate în FN3. Această decompoziție este astfel realizată încât aplicând joncțiunea naturală între proiecțiile obținute se reface relația inițială fără pierdere de informații.

Forma normală Boyce-Codd(FNBC)

Relația $r(a_1, a_2, \dots, a_n)$ este în FNBC dacă și numai dacă existența unei dependențe funcționale netrivială $X \rightarrow Y$,unde X și Y sunt seturi de atribute ale lui r implică existența dependențelor funcționale $X \rightarrow a_i$, pentru $i=1, 2, \dots, n$.

Această definiție poate fi reformulată utilizând termenul determinant.

Se numește determinant orice atribut simplu sau compus care implică funcțional alte atribute.

O relație este în FNBC dacă și numai dacă orice determinant al relației este cheie candidat.

Proprietăți :

1. Dacă o relație este în FNBC, ea este obligatoriu în FN3.
2. Există relații în FN3 care nu sunt în FNBC.
3. Dacă o relație nu este în FNBC, atunci această relație poate fi descompusă într-un set de proiecții FNBC astfel încât conținutul relației inițiale să poată fi refăcut fără pierdere de informație prin joncțiunea naturală a proiecțiilor.

Procesul de normalizare poate fi rezumat astfel :

N1. Se transformă tabelul inițial într-unul sau mai multe tabele cu valori atomice.

N2. Din relațiile de la pasul precedent se elimină toate dependențele funcționale parțiale obținând relații FN2.

N3. Din relațiile FN2 se elimină toate dependențele funcționale tranzitive de atribute cheie. Se obțin relații FN3.

N4. Se aleg proiecțiile relațiilor FN3 care elimină acele dependențe funcționale în care determinantul nu este cheie candidat și se obțin relații FNBC.

N5. Se aduc relațiile FNBC în FN4.

N6. Se aleg proiecțiile relațiilor FN4 care elimină dependențele joncțiune care nu sunt implicate prin chei candidat .Rezultă relații FN5.

5.2. Normalizarea bazei de date

Forma normala 1:

In aceasta etapa de normalizare se urmareste eliminarea grupurilor repetitive din fiecare tabela.

Analizand fiecare tabela se observa ca nu exista grupuri repetitive, deci baza de date este in forma normala 1 (1NF).

Forma normala 2:

O relatie este in 2NF daca este 1NF si orice atribut neprimitiv este complet dependent de orice cheie. Forma normala 2 se verifica doar la relatiile care au o cheie compusa pe pozitia de cheie primara. Daca pentru o relatie, cheia primara se compune dintr-un singur atribut atunci ea este in forma normala 2.

Tabelele bazei de date din sistemul nostrum au chei primare care se compun dintr-un singur atribut, in concluzie toate relatiile sunt deja in forma normala 2.

Forma normala 3

O relatie este in 3NF daca este in 2NF si nu exista nici un atribut care sa nu apartina cheii primare si care sa fie tranzitiv dependent de cheia primara.

Exprimand relatiile in forma normal de mai sus, observam ca nu exista dependente transitive. Deci relatiile sunt in forma normala 3.

Capitolul 6

Denormalizarea bazei de date

Denormalizarea unei BD reprezinta procesul invers operatiei de normalizare si duce la cresterea redundantei datelor. Prin aceasta se doreste, in principal, cresterea performantei si simplificarea programelor de interogare a datelor.

Obs.:

Denormalizarea se face numai dupa o normalizare corecta.

- Denormalizarea se face printr-o selectare strategica a structurilor care aduc avantaje semnificative

Denormalizarea trebuie insotita de masuri suplimentare de asigurare a integritatii datelor.

a) Cresterea performantei

Un caz frecvent intalnit in interogarea BD este cazul unor operatii sau calcule foarte des utilizate.

b) Simplificarea codului pentru manipularea datelor

Exemplu: Fie tabelul STOCURI utilizat de o firma pentru inregistrarea cantitatilor de materiale existente in fiecare din depozitele sale.

STOCURI (cod_depozit, cod_material, nume_material, cantitate)

Se cere aflarea tuturor depozitelor in care exista proteine.

Dupa normalizare avem:

STOCURI_1 (cod_depozit, cod_material, cantitate)

MATERIAL (cod_material, nume_material)

Interogarea in SQL va fi:

- varianta nenormalizata:

```
SELECT cod_depozit, cantitate
```

```
FROM stocuri
```

```
WHERE nume_material = "proteine";
```

- varianta normalizata:

```
SELECT cod_depozit, cantitate
```

```
FROM stocuri_1, material
```

```
WHERE stocuri_1.cod_material = material.cod_material
```

```
AND nume_material = "proteine";
```

O solutie care rezolva problema consta in a crea vederi bazate pe tabele normalizate.

Ex: CREATE VIEW stocuri AS

```
SELECT cod_depozit, stocuri_1.cod_material, nume_material,  
cantitate
```

```
FROM stocuri_1, material
```

```
WHERE stocuri_1.cod_material = material.cod_material;
```

Capitolul 7

SGBD-urile

7.1. Notiuni teoretice

- Sistemul de gestiune a bazelor de date reprezintă un pachet de programme care permit crearea, utilizarea și eliminarea obiectelor ce compun baza de date în scopul creșterii randamentului global al utilizării bazelor de date și având ca principal obiectiv reducerea dependenței aplicațiilor în raport cu structura datelor. Principala noutate introdusă de SGBD a fost separarea
- descrierii datelor de programul aplicativ. Descrierea datelor se face în mod unitar pentru toate aplicațiile cu ajutorul SGBD. SGBD-ul oferă mecanisme puternice de acces la date fără să necesite cunoștințe avansate privind structura datelor și sistemul de operare.
- Introducerea SGBD-ului a permis crearea unei arhitecturi pe 3 nivele a sistemului informatic:
- Nivelul logic corespunde modului în care fiecare utilizator vede conținutul bazei de date.
- Nivelul conceptual corespunde structurii întregii baze de date. Modelul conceptual este o descriere a conținutului bazei de date independentă atât de aplicațiile propriu-zise cât și de modelul de implementare al bazei de date.
- Nivelul intern se referă la modul de implementare al bazei de date.
- Modelul intern conține descrierea structurii de date implementate, descrierea fișierelor a tehnicilor de acces la date, lista utilizatorilor, drepturile lor de acces.
- Modelul relațional este un model logic al bazei de date care s-a impus datorită naturaleții sale și simplității în înțelegerea și manipularea structurilor de date. Reprezentarea unei baze de date sub formă de relații se numește model relațional al bazei de date.
- O relație este formată dintr-un tabel bidimensional în care coloanele corespund atributelor iar fiecare linie corespunde unei entități.
- Fiecare relație se identifică printr-un nume unic în cadrul unei baze de date.

- Fiecare atribut are un nume unic și trebuie să sugereze semnificația acestuia.
- O linie a relației se numește tuplu al relației .Prin relație materializăm o clasă de entități în care fiecare tuplu reprezintă o entitate.

```

Autocommit Display
CREATE TABLE Produs (id_produs varchar2(5), constraint id_produs_pk primary key (id_produs), nume_produs varchar2(20), descriere_scurta varchar2(100), pret_sugerat varchar2(20), unitate_de_vanzare varchar2(5) );

CREATE TABLE Regiune (id_regiune varchar2(5), constraint id_regiune_pk primary key (id_regiune), nume_regiune varchar2(20) );

CREATE TABLE Depozit (id_depozit varchar2(5), constraint id_depozit_pk primary key (id_depozit), adresa varchar2(50), stat varchar2(50), tara varchar2(50), oras varchar2(50), cod_zip number, numar_de_telefon number, id_regiune varchar2(5), foreign key (id_regiune) references regiune (id_regiune) );

CREATE TABLE Departament (id_departament varchar2(5), constraint id_departament_pk primary key (id_departament), nume_departament varchar2(20), id_regiune varchar2(5), foreign key (id_regiune) references regiune (id_regiune) );

CREATE TABLE Inventar (id_inventar varchar2(5), constraint id_inventar_pk primary key (id_inventar), cantitate_in_stoc varchar2(20), punct_de_ordine varchar2(20), maxim_in_stoc varchar2(20), stoc_epuizat varchar2(5), data_restocari date, id_produs varchar2(5), id_depozit varchar2(5), foreign key (id_produs) references produs (id_produs), foreign key (id_depozit) references depozit (id_depozit) );

CREATE TABLE Angajat (id_angajat varchar2(5), constraint id_angajat_pk primary key (id_angajat), prenume_angajat varchar2(20), nume_angajat varchar2(20), data_de_incepere date, titlu varchar2(20), salariu number, procent_comision number, comentari varchar2(100), id_departament varchar2(5), id_depozit varchar2(5), foreign key (id_departament) references departament (id_departament), foreign key (id_depozit) references depozit (id_depozit) );

insert into Produs (id_produs, nume_produs, descriere_scurta, pret_sugerat, unitate_de_vanzare) values (1,'benzi de alergat','foarte accesibila la pret si foarte compacta','600lei','buc' );
insert into Produs (id_produs, nume_produs, descriere_scurta, pret_sugerat, unitate_de_vanzare) values (2,'gantere','foarte confortabile si cu un numar foarte mare de discuri ','300lei','buc' );
insert into Produs (id_produs, nume_produs, descriere_scurta, pret_sugerat, unitate_de_vanzare) values (3,'bicicleta','dispune de 7 trepte de viteza','400lei','buc' );
insert into Produs (id_produs, nume_produs, descriere_scurta, pret_sugerat, unitate_de_vanzare) values (4,'proteine','anduranta si masa musculara in doar 3 saptamani de sala','80lei','buc' );
insert into Produs (id_produs, nume_produs, descriere_scurta, pret_sugerat, unitate_de_vanzare) values (5,'haltere','rezistente la shock si cu prindere anti-alunecare','500lei','buc' );

insert into Regiune (id_regiune, nume_regiune) values (1,'AMERICA DE NORD' );
insert into Regiune (id_regiune, nume_regiune) values (2,'AMERICA DE SUD' );
insert into Regiune (id_regiune, nume_regiune) values (3,'AFRICA' );
insert into Regiune (id_regiune, nume_regiune) values (4,'ASIA' );
insert into Regiune (id_regiune, nume_regiune) values (5,'EUROPA' );

insert into Depozit (id_depozit, adresa, stat, tara, oras, cod_zip, numar_de_telefon, id_regiune) values (1, 'BROOKLIN 34 ','CA','USA','SAN FRANCISCO',08540,0740011962,1 );
insert into Depozit (id_depozit, adresa, stat, tara, oras, cod_zip, numar_de_telefon, id_regiune) values (2, 'Dolce Gabana 21 ','PZ','Brazilia','State of Bahia',12572,0740157962,2 );
insert into Depozit (id_depozit, adresa, stat, tara, oras, cod_zip, numar_de_telefon, id_regiune) values (3, 'Uvuwue 3 ','CP','Africa','Zimbabwe',21152,0741215215,3 );
insert into Depozit (id_depozit, adresa, stat, tara, oras, cod_zip, numar_de_telefon, id_regiune) values (4, 'Xin Jin Ping 14 ','XA','China','Beijing',62181,0741468911,4 );
insert into Depozit (id_depozit, adresa, stat, tara, oras, cod_zip, numar_de_telefon, id_regiune) values (5, 'Liberatii 4 ','OT','Romania','Caracal',235200,0741111962,5 );

```

```

DROP TABLE Functie;
DESC depozit;
SELECT *FROM Loc_de_munca;
DELETE *FROM Angajat where Nume='Popescu' and Cod_functie=200222;
UPDATE Angajat set Nume='Georgescu' where Nume='Popescu' and Angajat_id=100200 where Angajat_id=200100;
SELECT Angajat_id,Nume,Prenume from Angajat

```

ID_PRODUS	NUME_PRODUS	DESCRIERE_SCURTA	PRET_SUGERAT	UNITATE_DE_VANZARE
1	benzi de alergat	foarte accesibila la pret si foarte compacta	600lei	buc
2	gantere	foarte confortabile si cu un numar foarte mare de discuri	300lei	buc
3	bicicleta	dispune de 7 trepte de viteza	400lei	buc
4	proteine	anduranta si masa musculara in doar 3 saptamani de sala	80lei	buc
5	haltere	rezistente la shock si cu prindere anti-alunecare	500lei	buc

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PRODUS	ID_PRODUS	Varchar2	5	-	-	1	-	-	-
	NUME_PRODUS	Varchar2	20	-	-	-	✓	-	-
	DESCRIERE_SCURTA	Varchar2	100	-	-	-	✓	-	-
	PRET_SUGERAT	Varchar2	20	-	-	-	✓	-	-
	UNITATE_DE_VANZARE	Varchar2	5	-	-	-	✓	-	-
1 - 5									

ID_REGIUNE	NUME_REGIUNE
1	AMERICA DE NORD
2	AMERICA DE SUD
3	AFRICA
4	ASIA
5	EUROPA

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
REGIUNE	ID_REGIUNE	Varchar2	5	-	-	1	-	-	-
	NUME_REGIUNE	Varchar2	20	-	-	-	✓	-	-
1 - 2									
ID_DEPOZIT	ADRESA	STAT	TARA	ORAS	COD_ZIP	NUMAR_DE_TELEFON	ID_REGIUNE		
1	BROOCKLIN 34	CA	USA	SAN FRANCISCO	8540	740011962	1		
2	Dolce Gabana 21	PZ	Brazilia	State of Bahia	12572	740157962	2		
3	Uvuwue 3	CP	Africa	Zimbabwe	21152	741215215	3		
4	Xin Jin Ping 14	XA	China	Beijing	62181	741468911	4		
5	Liberatii 4	OT	Romania	Caracal	235200	741111962	5		
5 rows returned in 0.00 seconds CSV Export									
Object Type: TABLE Object: DEPOZIT									
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPOZIT	ID_DEPOZIT	Varchar2	5	-	-	1	-	-	-
	ADRESA	Varchar2	50	-	-	-	✓	-	-
	STAT	Varchar2	50	-	-	-	✓	-	-
	TARA	Varchar2	50	-	-	-	✓	-	-
	ORAS	Varchar2	50	-	-	-	✓	-	-
	COD_ZIP	Number	-	-	-	-	✓	-	-
	NUMAR_DE_TELEFON	Number	-	-	-	-	✓	-	-
	ID_REGIUNE	Varchar2	5	-	-	-	✓	-	-
1 - 8									
ID_DEPARTAMENT	NUME_DEPARTAMENT	ID_REGIUNE							
1	AMERICA DE NORD	1							
2	AMERICA DE SUD	2							
3	AFRICA	3							
4	ASIA	4							
5	EUROPA	5							
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPARTAMENT	ID_DEPARTAMENT	Varchar2	5	-	-	1	-	-	-
	NUME_DEPARTAMENT	Varchar2	20	-	-	-	✓	-	-
	ID_REGIUNE	Varchar2	5	-	-	-	✓	-	-
1 - 3									
ID_INVENTAR	CANTITATE_IN_STOC	PUNCT_DE_ORDINE	MAXIM_IN_STOC	STOC_EPUIZAT	DATA_RESTOCARI	ID_PRODUS	ID_DEPOZIT		
1	59	1	100	nu	13-01-2012	1	1		
2	519	2	1000	nu	13-01-2012	2	2		
3	159	3	10100	nu	13-01-2002	3	3		
4	559	4	800	nu	13-01-2002	4	4		
5	119	5	124	nu	13-01-2002	5	5		

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
INVENTAR	ID_INVENTAR	Varchar2	5	-	-	1	-	-	-
	CANTITATE_IN_STOC	Varchar2	20	-	-	-	✓	-	-
	PUNCT_DE_ORDINE	Varchar2	20	-	-	-	✓	-	-
	MAXIM_IN_STOC	Varchar2	20	-	-	-	✓	-	-
	STOC_EPUIZAT	Varchar2	5	-	-	-	✓	-	-
	DATA_RESTOCARI	Date	7	-	-	-	✓	-	-
	ID_PRODUS	Varchar2	5	-	-	-	✓	-	-
	ID_DEPOZIT	Varchar2	5	-	-	-	✓	-	-
1 - 8									

ID_ANGAJAT	PRENUME_ANGAJAT	NUME_ANGAJAT	DATA_DE_INCEPERE	TITLU	SALARIU	PROCENT_COMISION	COMENTARI	ID_DEPARTAMENT	ID_DEPOZIT
1	Marinel	Columbeanu	13-09-2002	presedinte	1900	10	Comanda a fost deja plasata sper sa va bucurati	1	1
2	Costel	Meltenel	15-01-2006	distribuitor	1100	4	Comanda va venii in 2 zile	2	2
3	Francesca	Cambara	03-07-2012	resurse umane	1200	7	Angajatii sunt foarte productivi	3	3
4	Ionel	Popescu	01-11-2021	angajat	900	3	Este prima mea slujba si sunt deja foarte fericit	4	4
5	Alin	Dumitrescu	03-09-2010	gardian	100	1	Lucrez pentru prima data full time	5	5

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ANGAJAT	ID_ANGAJAT	Varchar2	5	-	-	1	-	-	-
	PRENUME_ANGAJAT	Varchar2	20	-	-	-	✓	-	-
	NUME_ANGAJAT	Varchar2	20	-	-	-	✓	-	-
	DATA_DE_INCEPERE	Date	7	-	-	-	✓	-	-
	TITLU	Varchar2	20	-	-	-	✓	-	-
	SALARIU	Number	-	-	-	-	✓	-	-
	PROCENT_COMISION	Number	-	-	-	-	✓	-	-
	COMENTARI	Varchar2	100	-	-	-	✓	-	-
	ID_DEPARTAMENT	Varchar2	5	-	-	-	✓	-	-
	ID_DEPOZIT	Varchar2	5	-	-	-	✓	-	-
1 - 10									

ID_CLIENT	NUME_CLIENT	ADRESA	ORAS	STAT	COD_ZIP	TARA	NUMAR_DE_TELEFON	ID_REGIUNE	ID_ANGAJAT
1	BIG JOHN	Cajaca 54	SAN FRANCISCO	CA	8540	USA	5299919282	1	1
2	WOMANSports	Camaca 76	SEATTLE	WASHINGTON	8541	USA	5299919281	1	1
3	Decathlon Alabama	Cambogia 5	Mobile	AI	7140	USA	5299919582	1	1
4	Decathlon Amazonas	Majamba 154	Manaus	SOA	15240	Brazilia	1499419782	2	2
5	Decathlon Bahia	Manamba 1154	Salvador	SOB	15157	Brazilia	1496411761	2	2
6	Decathlon Alagoras	Maxamba 153	Maceio	SOAL	15288	Brazilia	1392419482	2	2
7	Decathlon Zimbabwe	Najamba 254	Mutare	ZMB	22222	Zimbabwe	1499419781	3	3
8	Decathlon Zimbabwe	Naxamba 2154	Harare	ZMB	21212	Zimbabwe	1499419781	3	3
9	Decathlon Zimbabwe	Namamba 234	Gweru	ZMB	23232	Zimbabwe	1499419781	3	3
10	Decathlon Hiroshima	Nihon 1524	Hiroshima	HRS	32323	Japonia	1499419781	4	4

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0,00 seconds [CSV Export](#)

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CLIENT	ID_CLIENT	Varchar2	5	-	-	1	-	-	-
	NUME_CLIENT	Varchar2	20	-	-	-	✓	-	-
	ADRESA	Varchar2	50	-	-	-	✓	-	-
	ORAS	Varchar2	20	-	-	-	✓	-	-
	STAT	Varchar2	50	-	-	-	✓	-	-
	COD_ZIP	Number	-	-	-	-	✓	-	-
	TARA	Varchar2	50	-	-	-	✓	-	-
	NUMAR_DE_TELEFON	Number	-	-	-	-	✓	-	-
	ID_REGIUNE	Varchar2	5	-	-	-	✓	-	-
	ID_ANGAJAT	Varchar2	5	-	-	-	✓	-	-
									1 - 10

ID_COMANDA	DATA_COMENZI	DATA_EXPEDIERI	TIPUL_DE_PLATA	ID_CLIENT	ID_ANGAJAT
1	13-02-2001	15-02-2001	ramburs	5	1
2	15-05-2005	17-02-2005	card	7	2
3	06-09-2019	08-02-2019	ramburs	1	3
4	14-11-2009	16-11-2009	card	9	4
5	09-06-1999	10-06-1999	ramburs	15	5

5 rows returned in 0,00 seconds

[CSV Export](#)

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
COMANDA	ID_COMANDA	Varchar2	5	-	-	1	-	-	-
	DATA_COMENZI	Date	7	-	-	-	✓	-	-
	DATA_EXPEDIERI	Date	7	-	-	-	✓	-	-
	TIPUL_DE_PLATA	Varchar2	20	-	-	-	✓	-	-
	ID_CLIENT	Varchar2	5	-	-	-	✓	-	-
	ID_ANGAJAT	Varchar2	5	-	-	-	✓	-	-
									1 - 6

ID_OBIECT	PRET	CANTITATE	CANTITATE_EXPEDIATA	ID_PRODUS	ID_COMANDA
1	1800	57	3	1	1
2	30000	419	100	2	2
3	4800	147	12	3	3
4	4720	500	59	4	4
5	50000	19	100	5	5

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
OBIECT	ID_OBIECT	Varchar2	5	-	-	1	-	-	-
	PRET	Varchar2	20	-	-	-	✓	-	-
	CANTITATE	Varchar2	20	-	-	-	✓	-	-
	CANTITATE_EXPEDIATA	Varchar2	20	-	-	-	✓	-	-
	ID_PRODUS	Varchar2	5	-	-	-	✓	-	-
	ID_COMANDA	Varchar2	5	-	-	-	✓	-	-
									1 - 6

Capitolul 8

Concluzii

SGBD-ul folosit este cel de tipul relational deoarece prezinta cea mai simpla structura pe care o poate avea o baza de date .

Datele sunt organizate in tabele formate din inregistrari si campuri. In acest caz bazele de date relationale sunt foarte flexibile si usor de manuit.

Cele mai folosite baze de date sunt:

- Oracle

- SQL

Sistemele de gestiune a bazelor de date ofera o vedere externa a datelor, care nu se modifica atunci cand se schimba suportul de memorare fizic, ceea ce asigura imunitatea structurii bazei de date si a aplicatiilor la modificari ale sistemului hardware utilizat.

